Strategic Implementation of Ontology-Backed Natural Language Query Systems: Superiority Over Relational Metadata in the LLM Era

Interaction with structured enterprise data is currently undergoing a fundamental technological shift. For decades, extracting knowledge from relational databases required specialized expertise in Structured Query Language (SQL) or other formalized query languages. The advent of Large Language Models (LLMs) has drastically accelerated the development of Natural-Language-to-Query (NL2Query) interfaces, promising a seamless democratization of data access. However, deploying pure Text-to-SQL systems over large-scale, heterogeneous data landscapes frequently leads to massive errors in practice. When foundational models are exclusively confronted with unstructured relational metadata such as Data Definition Language (DDL) schemas, they suffer from high rates of syntactic hallucinations, semantic misinterpretations, and catastrophic execution failures in complex analytical queries[1].

To bridge the immense gap between human language and precise database execution, modern data architectures must evolve beyond pure relational definitions. The transition from merely exposing physical schemas to explicit semantic modeling represents the most critical challenge in Data Engineering today. By embedding domain knowledge into formalized ontologies and semantic layers, organizations can mathematically constrain the text generation of LLMs, enforce deterministic logic, and achieve extreme precision in query synthesis[4]. This detailed analysis evaluates the architectural vectors required for deploying robust NL2Query systems. It explicitly outlines where and why ontologies significantly outperform traditional relational metadata and presents highly efficient, practical implementation techniques (low-hanging fruits) for ecosystems based on the Resource Description Framework (RDF) and Labeled Property Graphs (LPG).

The Structural Deficit of Relational Metadata and the Limits of Text-to-SQL

The fundamental problem with zero-shot Text-to-SQL generation lies in the nature of the underlying data architectures: relational databases are primarily optimized for machine storage, normalization, and transactional integrity, not for semantic understanding by algorithms. When an LLM is presented with a raw SQL schema, the model must infer the complex business logic connecting isolated tables purely from context. In highly normalized database structures (such as Third Normal Form, 3NF), the conceptual meaning of an entity is often spread across dozens of tables, which are additionally labeled with cryptic identifiers that are barely human-readable[6].

Without explicit semantic guidance, the generative model is forced to guess the appropriate

table joins, filter conditions, and column mappings purely based on lexical similarities. This probabilistic guessing inevitably leads to the generation of queries that may be syntactically correct but are semantically entirely misleading. For instance, an LLM might correctly deduce an SQL join between a "Customer" table and an "Order" table, but incorrectly assume that the mere existence of a record equals realized revenue. Vital domain-specific status indicators like "order canceled" or "payment pending" are simply overlooked because the model cannot grasp the business context of the data[5].

How strongly LLMs depend on human-readable schemas and semantic clarity is demonstrated by an analysis of Text-to-SPARQL behavior across different knowledge graphs. When fine-tuning Llama-based models for text-to-SPARQL translation on DBpedia and Wikidata, significant discrepancies were observed. While the best model achieved a respectable Macro-F1 score of 60.68% on DBpedia—a graph with clearly named predicates—its performance collapsed to a mere 13.36% on Wikidata. This extreme discrepancy is directly attributed to the non-human-readable, numeric identifiers in Wikidata, whose meaning the model could not extract from pure context[8]. Such schema and identifier dependence highlights that relational metadata alone is insufficient to establish a generalizable NL2Query solution.

The Semantic Layer: Metrics vs. Complex Reasoning

To combat the ambiguity of raw relational metadata, the data analytics industry has widely adopted intermediary semantic layers, notably the "dbt Semantic Layer" alongside platforms like Cube or Malloy. These frameworks operate on a "Metrics-as-Code" philosophy. Instead of asking the LLM to piece together raw SQL from dozens of tables, data engineers proactively define critical business entities, dimensions, and measures in structured configuration files (usually YAML)[4].

When a natural language query is posed, the LLM's task is reduced to mapping the user's lexical intent to these predefined metric objects. A deterministic engine (e.g., MetricFlow) then takes over the compilation of the actual SQL code required to execute the query against the underlying data warehouse[12]. This hybrid approach of probabilistic intent recognition and deterministic code generation delivers outstanding results within its defined scope. Recent benchmark evaluations from 2026 demonstrate that models like GPT-5.3 Codex, in combination with the dbt Semantic Layer, achieve near 100% execution accuracy for modeled analytical queries. If a question falls outside the modeled scope, the system outputs precise error messages ("I cannot answer this question") instead of hallucinating[4].

Nevertheless, metric-focused semantic layers exhibit severe systemic limitations when highly complex, relationship-intensive logic queries are required. A semantic layer is built from the ground up to be execution-oriented; its primary purpose is standardizing aggregations and ensuring numerical consistency across different dashboards[9]. However, it lacks deep ontological mapping capabilities. When a user asks a complex "multi-hop" question requiring traversal across distant domain concepts—such as tracking a defective component in the supply chain through the manufacturing process to a specific group of end customers—the metric layer fails because it cannot compute the semantic path[9]. Furthermore, scaling the semantic layer requires massive manual modeling effort upfront: every new analytical question extending beyond the predefined YAML definitions must first be modeled by developers,

effectively just shifting the data delivery bottleneck from dashboard building to semantic modeling[4].

Comparison Dimension	Raw Text-to-SQL (DDL)	Semantic Layer (e.g., dbt)	Ontology (Knowledge Graph)
Query Accuracy (Standard)	Low to medium (~30-65%)[13]	Very high (~98-100%)	High (~85-95%)[15]
Query Accuracy (Multi-Hop/Complex)	Very low (< 20%)[6]	Not applicable (Fails)[10]	High (via traversal)
Hallucination Risk	Very high (wrong joins/logic)[16]	Low (deterministic rejection)[8]	Low (semantic constraint)
Initial Setup Effort	Low (direct connection)[8]	High (YAML definitions)[4]	Very high (Ontology design)[9]
Flexibility for Ad-hoc Questions	High (limited by accuracy)[12]	Low (strictly bound to metrics)	High (complete concept mapping)[5]

The Technological and Analytical Superiority of Ontological Context

Where relational metadata fails due to its lack of structure and metric-based semantic layers hit their limits due to a lack of flexibility, formalized ontologies unfold their full potential. An ontology is a highly expressive conceptual model that represents domain knowledge not as flat tables, but as a graph of interconnected entities, relationships, hierarchies, and logical axioms. Based on Semantic Web standards like the Web Ontology Language (OWL) and the Resource Description Framework (RDF), ontologies act as a digital twin of organizational reality, completely decoupled from the underlying physical database schema[9].

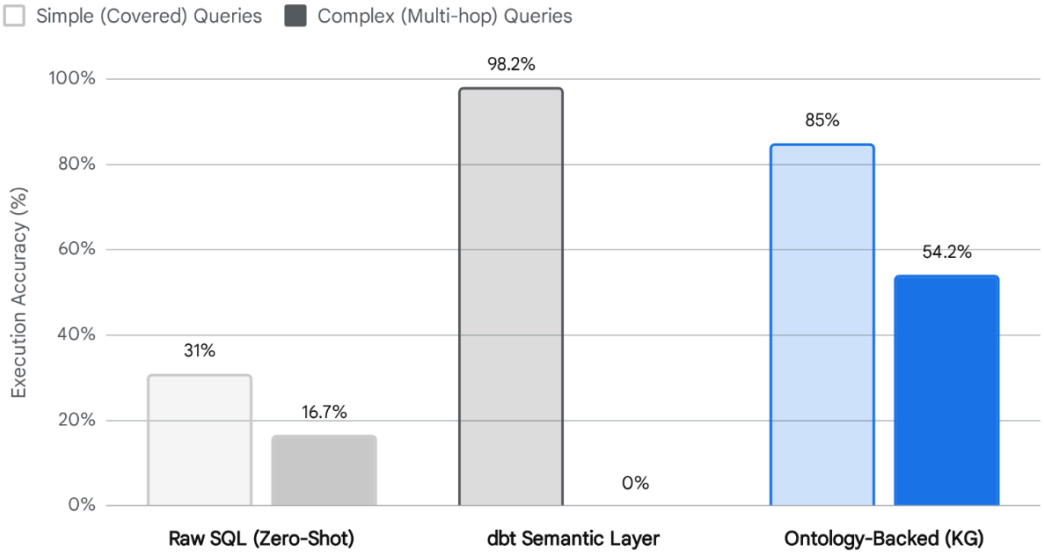
The decisive advantage of an ontology for NL2Query generation is that it provides the LLM with an explicit topological map of the business domain. Rather than confronting the model with abstract foreign keys (e.g., customer_id equals customer_id), an ontology precisely defines that a Customer *places* an Order, and an Order *contains* a Product. Moreover, ontologies support logical inference and transitive inheritance mechanisms. If the ontology defines that a Manager is a subclass of Employee, the LLM instinctively understands that any query regarding employees must also include managers without this needing to be explicitly programmed into the query[9].

Empirical evaluations confirm the massive performance gap between ontology-backed systems and pure relational pipelines. In evaluations over complex enterprise schemas from the insurance industry, the baseline performance of Text-to-SQL (using GPT-4) was a mere 16.7%. However, when the same generative model was provided with an ontological mapping of the domain instead of pure SQL DDL, query accuracy spiked to 54.2%—a massive leap in performance achieved solely by elevating the semantic context at the model level. Even more extensive analyses show that completely removing ontological annotations (such as domain, range, and comments in OWL representations) causes the accuracy of generated SPARQL code to plummet by up to 19 percentage points and SQL by 14 percentage points[22].

Query Accuracy by Metadata Provisioning Strategy

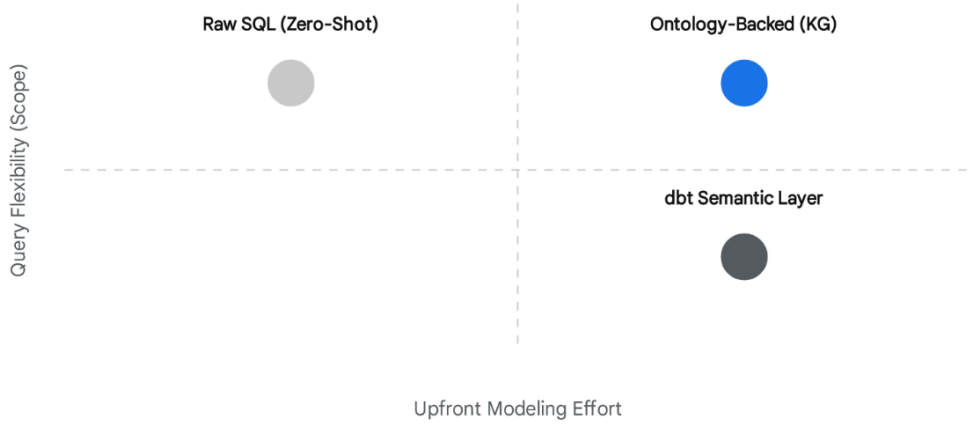
Comparing base execution accuracy and architectural trade-offs across three primary NLP-to-SQL paradigms.

Execution Accuracy by Query Complexity



Capability Scope vs. Maintenance Effort

Evaluating the trade-off between semantic flexibility and required upfront data modeling.



Evaluation of Natural Language Query accuracy across different metadata architectures. While deterministic semantic layers achieve near-perfect accuracy for pre-modeled metrics, ontology-driven systems provide the highest blended accuracy for complex, multi-hop domain queries.

Data sources: ResearchGate (data.world) , colrows.com (Spider 2.0) , dbt Labs , ResearchGate (NL2SQL Eval)

The semantic disambiguation provided by an ontology is the crucial factor. In legacy databases, a term like "revenue" could correspond to five different columns (gross revenue, net revenue, realized revenue, deferred revenue, projected revenue). By forcing organizations to map these terms cleanly and distinctly within an ontology, the LLM no longer has to guess the correct metric based on statistical word frequencies or unreliable table names. The ontology acts as a central Single Source of Truth, drastically narrowing the solution space for the model and thus guaranteeing consistent alignment with actual business definitions[5].

This fundamental advantage has led to the development of dedicated benchmarks to quantify the performance of LLMs in generating ontology-based queries. The "Spider4SPARQL" benchmark is a prime example of this development: it converts conventional SQL challenges into a semantic format based on an Ontology-Based Data Access (OBDA) approach. This benchmark includes thousands of novel SPARQL queries across cross-domain knowledge graphs and fixes data modeling errors of the original Spider datasets to test models fairly under the premise that semantic context knowledge is available[7]. Evaluations on such benchmarks, such as by the university's SPARQL-LLM system in the Text2SPARQL Challenge 2026, prove an increase in overall query accuracy by up to 59% over purely relational approaches, coupled with robust multilingualism[24].

Virtual Knowledge Graphs (VKG): Strategic Implementation without ETL

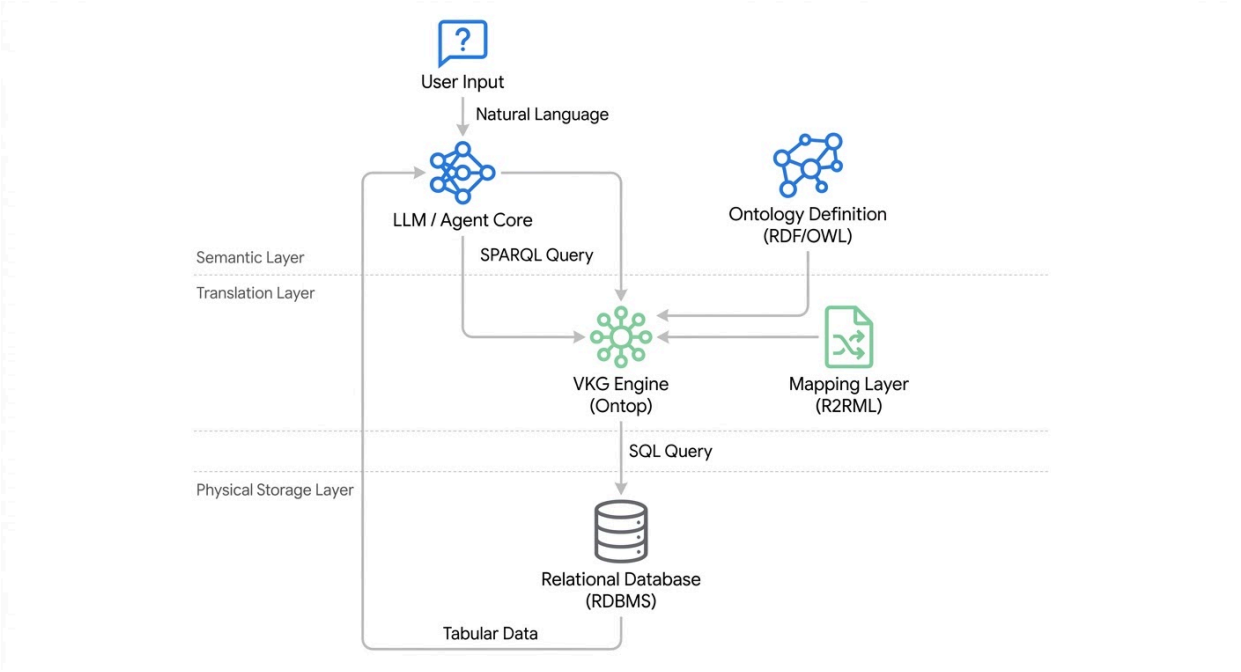
A historical and often severe obstacle to adopting ontologies in enterprise environments was the assumption that all operational data had to be extracted from high-performance relational databases and physically migrated into specialized graph databases (known as Triple Stores). This ETL (Extract, Transform, Load) process is extraordinarily expensive, leads to massive redundancies, disrupts operational workflows, and creates data synchronization latencies[25].

A highly pragmatic and resource-efficient "low-hanging fruit" implementation to bypass this hurdle is the Virtual Knowledge Graphs (VKG) architecture. Systems like *Ontop* leverage the principles of Ontology-Based Data Access (OBDA) to expose arbitrary relational databases as virtual RDF knowledge graphs at runtime[18]. In this approach, the raw data remains entirely in its native, relational infrastructure (e.g., in PostgreSQL instances, Amazon Athena, or Snowflake). Instead of moving data, Data Engineers create purely declarative mappings—mostly using the W3C standard R2RML (RDB to RDF Mapping Language)—that establish an explicit connection between the tables and columns of the relational database and the classes and properties of the ontology[18].

The process flow is elegant and highly efficient: When an LLM translates a natural language request into a SPARQL query based on the ontological context, the Ontop engine intercepts this SPARQL query. Using the R2RML mappings, Ontop rewrites the SPARQL query on-the-fly into a highly optimized SQL query[25]. This dynamically generated SQL query is then pushed down for execution to the underlying relational database, after which the tabular results are returned and seamlessly formatted as graph data to the end user or agent[25].

This architecture offers immense strategic value. The LLM interacts exclusively with the clean, semantically deep ontology and remains completely isolated from the often chaotic and

inconsistent physical structure of the legacy database[27]. Should the underlying database schema change, developers merely adjust the R2RML mappings; the ontology itself and the LLM's system prompts remain entirely untouched. This consistent decoupling of the logical meaning layer from the physical storage layer represents a significant structural advantage over traditional Text-to-SQL methods[18].



Additional implementations demonstrate the practical viability of this concept in cloud-native architectures. For example, orchestrating a VKG with Amazon Neptune to store the OWL ontology and Amazon Athena to execute the translated SQL queries enables a robust agent ecosystem incorporating "fail-soft" mechanisms[28]. This separation ensures that LLMs only come into contact with domain-specific business terms and that generation occurs at a higher level of abstraction (SPARQL), which is much closer to the semantic structures of natural language than relational schemas.

The Role of Labeled Property Graphs (LPG) and Text-to-Cypher

While the Resource Description Framework (RDF) and the SPARQL query language dominate the standardized Semantic Web and the scientific ontology sector, the rapidly growing market for enterprise internal graph databases is largely shaped by Labeled Property Graphs (LPGs) like Neo4j, Memgraph, and Amazon Neptune[29]. The LPG data model differs significantly from RDF on a conceptual level: Instead of relying on strict subject-predicate-object triples, LPGs allow both nodes and the edges connecting them to carry complex internal key-value properties. This characteristic makes LPGs extremely powerful for deep network traversals, algorithmic graph analytics, and real-time transactional analytics[31]. The de-facto standard languages for querying LPGs are Cypher and the recently adopted international ISO standard Graph Query Language (GQL)[29].

Mastering the "Text-to-Cypher" (or "Text-to-GQL") task thus represents an essential capability for future-proof enterprise-wide NL2Query systems. The problem, however, is that translating natural language into valid Cypher code poses a much steeper learning curve and higher hurdle for LLMs than SQL generation. Because Cypher or GQL source code is present in much smaller volumes in global training corpora (like GitHub or StackOverflow) compared to ubiquitous SQL, foundation models inherently struggle with the correct syntax for graph traversals[2].
Attempting zero-shot Text-to-Cypher over uncleaned, complex enterprise schemas often results in disastrous execution rates. Comprehensive benchmark evaluations (such as CypherBench, which tests Text-to-Cypher across billions of records and complex multi-hop scenarios) show that even the most advanced proprietary models (like Claude 3.5 Sonnet) only peak at around 61.58% accuracy. Smaller open-source models with fewer than 10 billion parameters frequently drop below a 20% success rate on Text-to-Cypher tasks[13]. This generative difficulty is exacerbated by the fact that LPGs, unlike relational databases, often do not enforce strict type constraints or standardized unit values at the schema level, leaving LLMs with additional room for interpretation.

The Need for Multi-Agent GraphRAG Architectures

To compensate for the systemic fragility of zero-shot Text-to-Cypher, cutting-edge research is rapidly moving towards specialized multi-agent architectures, often packaged within Retrieval-Augmented Generation (GraphRAG) frameworks. Rather than relying on a single, monolithic call to an LLM designed to translate the user query into a final Cypher query in one pass (single-shot), the entire workflow is broken down into autonomous subtasks. Each step is handled by a dedicated LLM agent or a rule-based module[30].
A robust multi-agent GraphRAG pipeline typically consists of the following interlocking components:

1. **Query Generator:** This agent formulates an initial draft of the Cypher query based on the user's natural language input. To minimize hallucinations, it is strictly grounded by a locally injected graph schema containing descriptions of relevant node labels and permissible relationship types[30].
2. **Named Entity Extractor:** This independent AI layer analyzes the newly generated Cypher query before it is executed. Its goal is to identify literal values, specific node labels, and property keys that could potentially have been hallucinated by the LLM[2].
3. **Verification Module:** This component programmatically validates the entities isolated by the extractor against the actual structure of the graph database. If an entity is not found (e.g., if the LLM invents a node label Mitarbeiter although the database requires the label Employee), the module searches the database for syntactically similar alternatives (e.g., using Levenshtein distance metrics) and provides the agent ecosystem with a validated replacement[30].
4. **Query Evaluator:** This agent acts as a semantic critic. It compares the finally executed data against the original intent of the user question and checks whether the chosen traversal path logically answers the request. If the data structures are empty or obviously illogical (e.g., overly restrictive WHERE clauses), the evaluator formulates dedicated revision instructions (via an "Instructions Generator") and sends them back to the Query

Generator for a new pass[30].

By employing an iterative, stateful verification loop, this multi-agent architecture systematically eliminates the primary sources of error in LLM graph queries: hallucinated schema elements, incorrect graph traversal directions, and invalid filter conditions. The deep integration of a verification step guarantees that no blindly hallucinated Cypher query is ever executed directly and unchecked against the production database, drastically increasing stability for the end user[30].

Highly Effective Implementation Techniques (Low-Hanging Fruits) for Optimization

The transition from theoretical architectural models to production-ready NL2Query deployments absolutely requires overcoming the context window limits of LLMs while ensuring absolute syntax validity. In recent developments, several highly actionable techniques have emerged that massively stabilize both Text-to-SPARQL and Text-to-Cypher pipelines without requiring expensive fine-tuning or model retraining.

1. Dynamic Schema Pruning (Context Minimization)

A major vulnerability in generative query creation is the limited capacity (Context Window) of LLMs. Attempting to embed an entire enterprise database schema or a sprawling multi-domain ontology into the prompt is not only computationally inefficient and expensive (in terms of token costs), but it also degrades the model's attention mechanisms (Attention Degradation), paradoxically leading to more hallucinations[6].

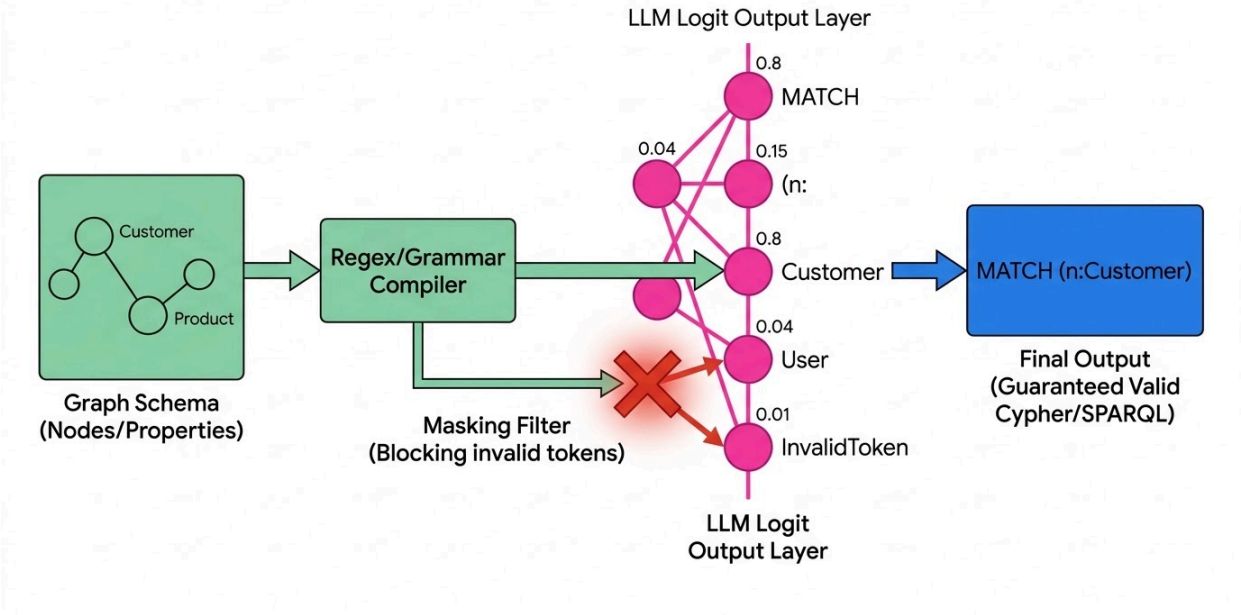
The optimal technical solution for this is dynamic "Schema Pruning". Instead of injecting the entire graph definition, these systems extract only the structural metadata that is strictly relevant to the user's specific query. In RDF/SPARQL environments, this is elegantly achieved by utilizing the "Vocabulary of Interlinked Datasets" (VoID) as well as "Shape Expressions" (ShEx) or the "Shapes Constraint Language" (SHACL)[8].

VoID provides a compact statistical summary of the structural partitions of a graph (e.g., class and predicate frequencies), giving the LLM an exact understanding of which predicates are even available without having to process millions of raw triples[8]. SHACL and ShEx, on the other hand, act as rigid validation schemas. By extracting a specific, localized SHACL Shape that perfectly matches the entities identified in the user request, the LLM receives explicit, compressed rules about what properties an entity *must* possess and what data types are permissible for them[38]. This precisely narrowed context extremely reduces the search space for the generative model, conserves token bandwidth, and maximizes accuracy. In evaluations within complex power grid domains, such adaptive metadata pruning more than doubled the successful SQL execution rate from 40.2% to 84.6%.

2. Grammar-Constrained Decoding

Perhaps the most powerful and yet fastest-to-implement measure (low-hanging fruit) in the current AI engineering landscape is grammar-constrained decoding. Generative language models operate on the fundamental basis of probability: they calculate the statistically most likely next token. Left entirely unconstrained, the model can and inevitably will generate

arbitrary text that violates the strict formal syntax rules of SPARQL, Cypher, or SQL[2]. Grammar-constrained decoding intervenes directly at the model's inference layer. By using open-source libraries like Outlines, Guidance, or through direct integration into high-performance servers like vLLM, engineers can rigidly bound the mathematical token generation process[3].



The mechanism operates by compiling the entire permissible graph schema—i.e., all valid node labels, relationship types, and property keys—into a massive Regular Expression (Regex) or a Context-Free Grammar (CFG) beforehand[3]. During inference, specifically at every computational step where the LLM calculates the probabilities (logits) for the next word, the decoding engine evaluates these tokens against the compiled regex. Any token that would violate Cypher syntax, or that points to a node label that does not exist in the schema, is assigned an absolute probability of zero, ensuring it can never be generated.
This technique brilliantly unites the linguistic flexibility of neural networks with the strictly deterministic rules of symbolic programming (Neuro-Symbolic AI). In practice, grammar-constrained decoding results in a 100% Syntactic Validity Rate (SVR). Developers no longer need to beg the model via "Prompt Engineering" to "please output valid Cypher"; the impossibility of syntax errors is mathematically guaranteed at the hardware level. Evaluations in the realm of Building Information Modeling (BIM) have proven that applying grammar constraints to relatively small open-weight models (e.g., with 7 billion parameters) literally doubles their execution accuracy, enabling them to compete with much more massive, unconstrained proprietary models.

3. Iterative Tool Use (MCP) instead of Monolithic Generation

Early implementations of NL2Query relied heavily on highly autonomous tools (High-Agency Tools). An LLM was handed a single, powerful function (e.g., execute_sql()) with the expectation

that the model would formulate the entire, complex query in a single shot. This monolithic approach has proven to be highly error-prone in complex enterprise domains. The new industry best practice is the use of iterative toolchains with low autonomy, often implemented via the Model Context Protocol (MCP).

Instead of demanding the immediate generation of the query, the LLM is equipped with granular discovery tools. These include functions like get_entity_properties(), explore_neighborhood(), or validate_pattern(). The agent uses these tools to explore the ontology step by step, construct the query block by block, and validate intermediate logical hypotheses. Because these tools modify a persistent status (Stateful Workflow), the LLM receives immediate feedback if it takes a wrong path in the ontology. This allows for an instant rollback of the assumption and a correction, without needing to abort and restart the entire generation process. This cyclical way of working shifts the burden from mere zero-shot memorization in the LLM to systemic, exploratory reasoning at runtime.

Sector-Specific Use Cases: Accelerating the Path to Value Creation

The strategic advantage of ontology-backed LLM queries is most evident in industries characterized by historically grown, extremely nested, and profound hierarchical structures. Organizations do not have to reinvent the wheel or design ontologies from scratch to generate value here; the adaptation of established, industry-wide frameworks provides an immediate foundation for data.

In the Architecture, Engineering, and Construction (AEC) sector, the integration of the "Brick Ontology" offers an explicit, RDF-based schema for mapping the physical and logical components of smart buildings—from complex HVAC (heating, ventilation, air conditioning) systems to abstract spatial hierarchies[43]. By implementing Text-to-SPARQL over the Brick Ontology, facility managers can query highly complex sensor dependencies using natural language instead of having to dig through unstandardized, raw telemetry data[43].

Analogous advancements can be seen in the biomedical sector and healthcare. Here, foundational ontologies like SNOMED CT or the Gene Ontology provide massive, pre-compiled networks of clinical and biological relationships[47]. Integrating these existing ontological standards into a multi-agent generation pipeline allows research institutions to bypass the massive modeling bottlenecks that cripple traditional semantic dashboard deployments.

Another often overlooked but highly lucrative enterprise application lies in the direct integration of these semantic querying capabilities into the Software Development Life Cycle (SDLC). Agent-based systems use enterprise ontologies to steer the automated creation of technical documentation. By guiding the LLM through an explicit operational ontology, the generated documentation reflects the true dependencies mapped in code and API interfaces, instead of relying on inconsistent developer comments or outdated manual specifications[49].

Conclusion and Strategic Recommendations

The era in which relational databases were meant to be queried exclusively through uncontrolled zero-shot Text-to-SQL models has proven insufficient for complex enterprise

environments. While exposing raw schemas leads to catastrophic hallucinations and metric-based semantic layers lack the flexibility needed for deep multi-hop analytics, ontologies represent the perfect middle ground. They provide the necessary topological context to effectively steer and constrain Large Language Models.
To maximize the efficiency, precision, and reliability of Natural-Language-to-Query systems, leaders in Data Engineering should implement the following strategic recommendations:

1. **Move away from pure Text-to-SQL for complex analytics:** Accept the realization that the mathematical and semantic ambiguity inherent in highly normalized relational schemas (like 3NF) poses an almost insurmountable barrier to the accuracy of zero-shot LLMs. Investments in pure prompt engineering approaches at the relational level yield no measurable success beyond a certain point (diminishing returns).
2. **Introduce Virtual Knowledge Graphs (VKG) as an integration bridge:** Avoid the immense infrastructural costs associated with migrating terabytes of relational data into graph databases. Utilize established OBDA frameworks like Ontop to virtually link relational tables with ontological classes via R2RML. This enables the execution of semantically grounded SPARQL queries directly against existing high-performance RDBMS.
3. **Mandatory implementation of grammar-constrained decoding:** Immediately integrate libraries like Outlines or vLLM to enforce strict syntax validation at the token level for Cypher and SPARQL generations. Eliminating syntactic failure at the model inference layer is the most effective and cost-efficient method (low-hanging fruit) to stabilize query pipelines and enable the use of smaller, cost-efficient open-source models.
4. **Adapt Multi-Agent Verification for GraphRAG:** Say goodbye to monolithic, single-step query generation. Deploy agent-based architectures that decouple the process into specialized subtasks: query generation, entity extraction, validation against the physical database schema, and logical evaluation of the execution results (Evaluator critique).
5. **Consistent use of Schema Pruning:** Protect the sensitive context window of LLMs by dynamically pruning metadata. Instead of injecting entire data dictionaries into prompts, only pass the structural metadata (for instance, extracted SHACL Shapes and VoID statistics) that is absolutely necessary for the currently active user request.

By deeply embedding explicit semantic structures into the generation pipeline, organizations transform their AI implementations from probabilistic, often faulty guessing machines into deterministic, auditable, and structurally flawless execution environments for enterprise knowledge.

Works cited

1. (PDF) A Text-to-SQL strategy based on large language models and, https://www.researchgate.net/publication/400912788_A_Text-to-SQL_strategy_based_on_large_language_models_and_knowledge_graphs_for_real-world_databases
2. How Can LLMs Turn Text Into Cypher in Complex Knowledge,

https://ieeexplore.ieee.org/iel8/6287639/10820123/10990239.pdf
3. IDS-Constrained Text-to-Graph Querying for IFC Models, https://re.public.polimi.it/retrieve/759cdff6-83df-4a48-baff-78e612f5c392/EC32026_306.pdf
4. SQL Alternative: Languages, Then Agents - InfiniSynapse, https://infinisynapse.com/guides/text-to-sql-alternative
5. Why ontology for text-to-SQL? - Nicholas Chen, https://nicholaschen.me/blogs/ontology-text-to-sql
6. Text-to-SQL Empowered by Large Language Models: A Benchmark, https://www.researchgate.net/publication/380309883_Text-to-SQL_Empowered_by_Large_Language_Models_A_Benchmark_Evaluation
7. Natural Language Access to Domain-Specific Metadata:A Reusable, https://arxiv.org/html/2607.18029v1
8. Enabling Knowledge Graph Understanding at Scale with the ... - arXiv, https://arxiv.org/html/2609.11569v1
9. Ontology vs. Semantic Layer: Why Your AI Agent Needs Both, https://medium.com/oceanbase-database/ontology-vs-semantic-layer-why-your-ai-agent-needs-both-a2e24c8060a1
10. Timbr vs dbt Semantic Layer, https://timbr.ai/timbr-vs-dbt-semantic-layer/
11. The Semantic Layer That Travels With the Data - My Framer Site, https://www.nx1.io/blog/business-semantic-layer
12. Semantic Layer vs. Text-to-SQL: 2026 Benchmark Update - dbt Docs, https://docs.getdbt.com/blog/semantic-layer-vs-text-to-sql-2026
13. Semantic Layer vs. Knowledge Graph: Key Differences (2026), https://colrows.com/blogs/semantic-layer-vs-knowledge-graph/
14. Ontology vs. Semantic Layer: Differences & How to Choose (2026), https://atlan.com/know/ontology-vs-semantic-layer/
15. A Benchmark to Understand the Role of Knowledge Graphs on, https://www.researchgate.net/publication/381292240_A_Benchmark_to_Understand_the_Role_of_Knowledge_Graphs_on_Large_Language_Model's_Accuracy_for_Question_Answering_on_Enterprise_SQL_Databases
16. Structured Agentic Retrieval over Knowledge Graphs with MCP and, https://www.eurecom.fr/publication/8936/download/data-publi-8936.pdf
17. LLM-supported collaborative ontology design for data and ... - Frontiers, https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2025.1676477/full
18. Accessing Semi-structured Data with RML and LLMs, https://kar.kent.ac.uk/114635/1/dl25.pdf
19. An Introduction to Ontology Engineering - KR 2016, https://people.cs.uct.ac.za/~mkeet/files/OEbookV1.pdf
20. Ontop: Answering SPARQL queries over relational databases, https://www.researchgate.net/publication/304352133_Ontop_Answering_SPARQL_queries_over_relational_databases
21. IJCAI-ECAI2026 PREPRINT - AWS, https://ijcai-preprints.s3.us-west-1.amazonaws.com/2026/1984.pdf

22. A Reusable Framework for LLM Query Generation - arXiv, https://arxiv.org/pdf/2607.18029
23. Spider4SPARQL: A Complex Benchmark for Evaluating Knowledge, https://arxiv.org/html/2309.16248v2
24. SPARQL-LLM in the TEXT2SPARQL 2026 Challenge - arXiv, https://arxiv.org/html/2610.13648v1
25. Spider4SPARQL: A Complex Benchmark for Evaluating Knowledge, https://www.computer.org/csdl/proceedings-article/bigdata/2023/10386182/1TUPOoXyJUs
26. Ontop - INODE, https://www.inode-project.eu/post/ontop.html
27. Building a semantic layer for structured data on AWS - Medium, https://medium.com/@fhuthmacher/building-a-semantic-layer-for-structured-data-on-aws-cdf55a8859f2
28. GitHub - aws-samples/sample-semantic-layer-structured, https://github.com/aws-samples/sample-semantic-layer-structured
29. Cypher: An Evolving Query Language for Property Graphs, https://www.researchgate.net/publication/325371427_Cypher_An_Evolving_Query_Language_for_Property_Graphs
30. Multi-Agent GraphRAG: A Text-to-Cypher Framework for Labeled, https://arxiv.org/html/2511.08274v1
31. 50 Knowledge Graph Interview Questions and Answers for 2026, https://cloudsoftsol.com/blog/knowledge-graph-interview-questions/
32. Knowledge Graphs - Complete Guide - Data AI Hub, https://www.dataaihub.co/learn/knowledge-graphs
33. BIMConverse - GraphRAG for IFC Natural Language Queries, https://blog.iaac.net/bimconverse-graphrag-for-ifc-natural-language-queries/
34. Text2Cypher - GraphRAG, https://graphrag.com/reference/graphrag/text2cypher/
35. CypherBench: Towards Precise Retrieval over Full-scale Modern, https://arxiv.org/html/2412.18702v1
36. Multi Agent RAG | PDF | Information Retrieval | System - Scribd, https://www.scribd.com/document/1069329068/Multi-Agent-RAG
37. sparql-llm - PyPI, https://pypi.org/project/sparql-llm/
38. Your LLM Is Guessing. Here's the Architecture That Makes It Stop, https://medium.com/@roanmonteiro/your-llm-is-guessing-heres-the-architecture-that-makes-it-stop-b60247be59e7
39. Real-Time SPARQL Query Generation from Natural Language, https://arxiv.org/html/2512.14277v1
40. AllegroGraph 9.0.3 Natural Language SPARQL Queries - Franz Inc., https://franz.com/agraph/support/documentation/natural-language-sparql-queries.html
41. Mohamed Anis Koubaa / Knowledge Management System · GitLab, https://gitlab.kit.edu/koubaa/knowledge-management-system
42. Daily Papers - Hugging Face, https://huggingface.co/papers?q=structured%20RAG
43. (PDF) Build2SPARQL: A Large-Scale Text-to-SPARQL Benchmark,

https://www.researchgate.net/publication/415152618_Build2SPARQL_A_Large-Scale_Text-to-SPARQL_Benchmark_Dataset_for_Building_Knowledge_Graph_Querying
44. OntoSage: Intelligent Human-Building Smartbot for Semantic Smart, https://orca.cardiff.ac.uk/id/eprint/183621/7/s11280-026-01403-0.pdf
45. The Resource Description Framework (RDF) and its Vocabulary, https://www.researchgate.net/publication/255665214_The_Resource_Description_Framework_RDF_and_its_Vocabulary_Description_Language_RDFS
46. Build2SPARQL: A Large-Scale Text-to-SPARQL Benchmark Dataset, https://arxiv.org/html/2610.00224v1
47. Large Language Models for Ontology Engineering: A Systematic, https://www.semantic-web-journal.net/system/files/swj4001.pdf
48. Improving GPT-Driven Medical Question Answering Model Using, https://www.mdpi.com/2079-9292/14/17/3488
49. Agentic AI for Technical Documentation: A real-world, experience, https://medium.com/@nc_mike/agentic-ai-for-technical-documentation-a-real-world-experience-based-review-4cb8ac56b6e6